

Fudan University

復旦大學

Project1 MNIST 分类

实验报告

方昱凯 23300180045

2026 年 5 月 8 日

# 1 实验目的

在 MNIST 上实现并训练 **MLP** 与 **CNN**，比较二者在相同参数设定下在验证集与官方测试集上的表现。之后进行 **Part C**：恒定学习率作为基线、加入余弦退火学习率、加入 Dropout 正则，分别在 MLP 与 CNN 上观察趋势。

## 2 数据及其划分

- **训练图像**：train-images-idx3-ubyte.gz，标签 train-labels-idx1-ubyte.gz。
- **官方测试集**：t10k-images-idx3-ubyte.gz / t10k-labels-idx1-ubyte.gz
- **划分方法**：打乱 60000 个数据后，取前 10000 作为 **dev** 验证，后 50000 用于训练；由随机数种子与 idx.pickle（划分方法）固定。在 Part B/C 使用 `--reuse_idx` 参数保证与 Part A 使用同一划分。

### 2.1 MLP (Part A)

- MLP 模型结构：**784**  $\rightarrow$  **600**  $\rightarrow$  **10**，激活函数选择 ReLU，损失函数选择交叉熵 (MultiCrossEntropyLoss)。



- 优化：SGD， $lr = 0.06$ ，无学习率调整，无 Dropout。

### 2.2 CNN (Part B)

- 结构见 mynn/models.py 中 Model\_CNN



- 优化设定与 Part A 对齐（确保公平对比），仅模型不同。

## 2.3 监控与保存

- 可调参数 `eval_interval=100`: 每 100 个 iteration 在 dev 上进行一次评估
- 以 dev 准确率最高保存 `best_model.pickle`
- 参数 `--save_fig` 用于保存曲线 PNG;
- 参数 `--save_metrics` 用于保存 `.npz` (包含 `best_score`; 若开参数 `--eval_test` 则还包含 `test_acc/test_loss`)。

## 3 实验配置表

| 项目                         | Part A (MLP)             | Part B (CNN)             | Part C (每组)                  |
|----------------------------|--------------------------|--------------------------|------------------------------|
| Epoch                      | 8                        | 8                        | 10 (默认)                      |
| Batch size                 | 64                       | 64                       | 64                           |
| 优化器                        | SGD, $lr=0.06$           | 同左                       | SGD, $lr=0.1$                |
| Seed                       | 309                      | 309                      | 64                           |
| Scheduler                  | none                     | none                     | baseline: none; cosine       |
| Dropout                    | 0                        | 0                        | baseline/cosine: 0; 正则组: 0.3 |
| <code>eval_interval</code> | 100                      | 100                      | 100                          |
| <code>log_iters</code>     | 100                      | 100                      | 100                          |
| 测试集                        | <code>--eval_test</code> | <code>--eval_test</code> | <code>--eval_test</code>     |

## 4 实验结果

### 4.1 Part A / Part B (MLP vs CNN)

表 1: MLP vs CNN

| 模型  | Best dev acc | Test acc | Test loss |
|-----|--------------|----------|-----------|
| MLP | 94.59%       | 94.88%   | 0.97236   |
| CNN | 98.35%       | 98.18%   | 0.05337   |

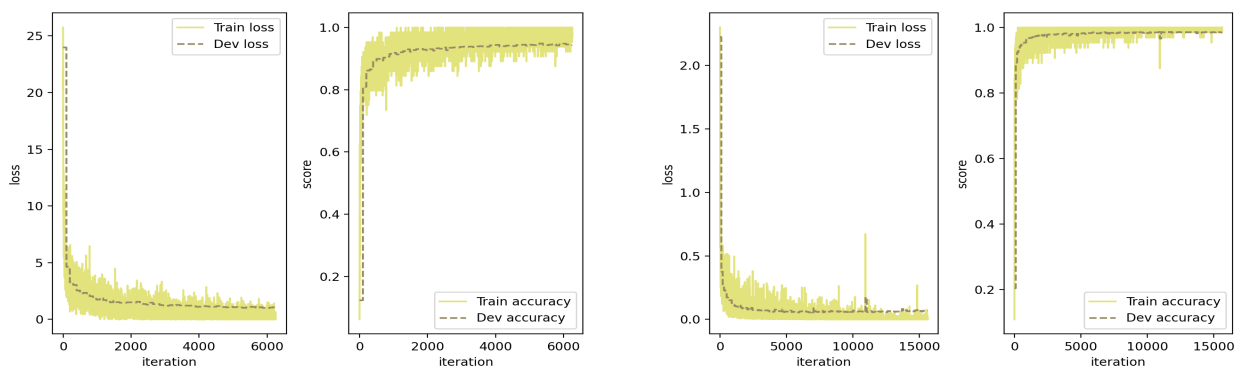


图 1: Part A (左) / Part B (右) 训练曲线

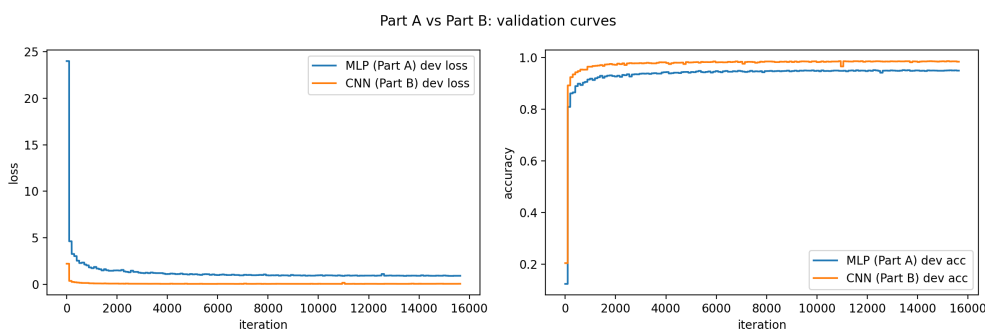


图 2: MLP vs CNN 验证曲线叠图

**简要分析:** 在相同参数设置下, CNN 相较于 MLP 来说收敛速度更快, 而且收敛后效果更好, 精度更高。主要原因应该是 CNN 能更好地利用图像的空间结构, 并且能更好的适应非线性映射, 所以在训练集、验证集以及测试集上的表现都优于 MLP。

## 4.2 Part C (组内对比)

**选取方向一:** 学习率调度, 选取余弦退火方法调节学习率

**选取方向二:** 随机 Dropout, 默认  $p=0.3$  (MLP 在; CNN 在卷积层后的全连接层加入 Dropout)

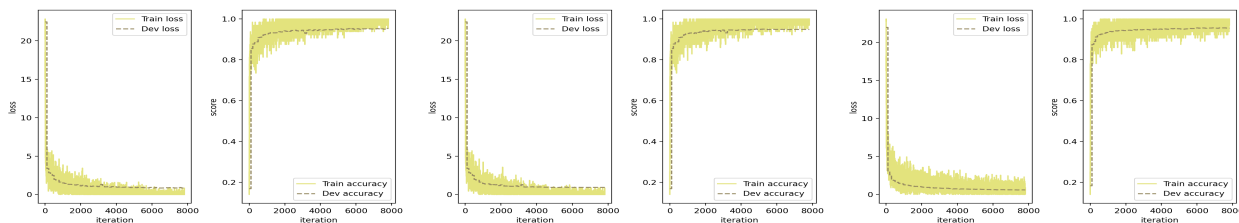


图 3: MLP 训练曲线: baseline: 左/余弦学习率: 中/Dropout: 右

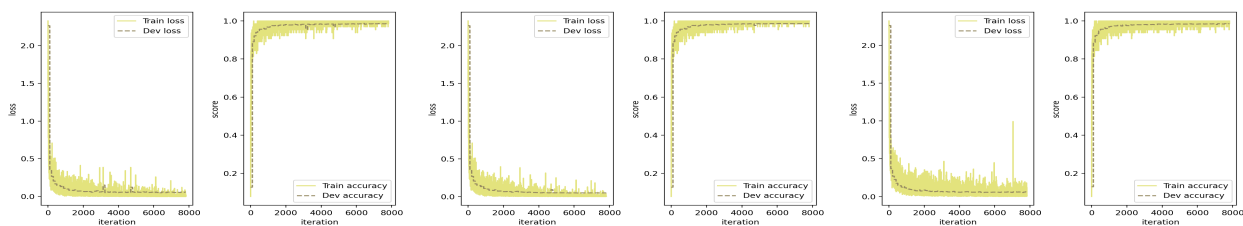


图 4: CNN 训练曲线: baseline: 左/余弦学习率: 中/Dropout: 右

表 2: Part C — MLP

| 设置          | Dev (best) | Test acc | Test loss |
|-------------|------------|----------|-----------|
| Baseline    | 95.33%     | 95.26%   | 0.8       |
| Cosine LR   | 94.90%     | 94.69%   | 0.95299   |
| Dropout 0.3 | 95.65%     | 95.22%   | 0.67487   |

表 3: Part C — CNN

| 设置          | Dev (best) | Test acc | Test loss |
|-------------|------------|----------|-----------|
| Baseline    | 98.56%     | 98.56%   | 0.05375   |
| Cosine LR   | 98.59%     | 98.61%   | 0.04524   |
| Dropout 0.3 | 98.51%     | 98.62%   | 0.04779   |

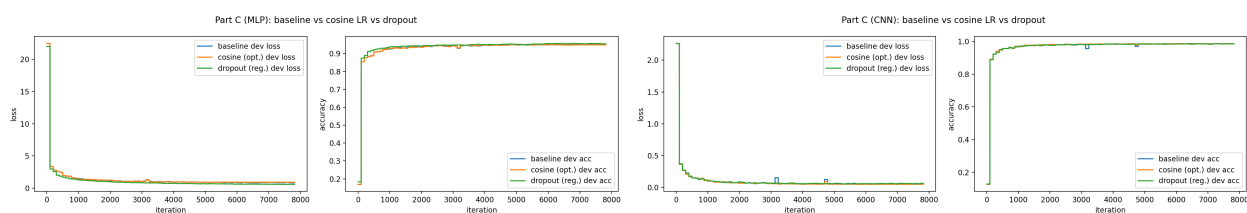


图 5: Part C 验证曲线 (MLP: 左 / CNN: 右)

**简要分析:** 添加学习率调度可以使训练更快达到收敛, 加入 Dropout 策略可以更好的防止模型过拟合。根据训练结果以及训练过程发现在添加了学习率调度或者 Dropout 后, 效果与未添加基本无异甚至略差, 其原因很可能是因为这个分类任务过于简单, 从上图中可以发现训练在经过 1000 iterations 后基本已经趋于稳定达到饱和, 后续产生的波动属于训练过程的随机误差, 所以导致实验结果无法体现加入优化后的优势。

## 5 结论

**PartA、PartB:** 在相同随机种子、相同 train/dev 划分、相同批量与学习率等条件下, CNN 通常能更好利用 MNIST 图像的局部空间结构, 验证集与测试集上的表现总体优于全连接 MLP, 这与卷积更适合图像分类的预期一致。

**PartC:** 1. 使用余弦退火 (cosine) 学习率调度相对恒定学习率的 baseline, 可以带来更平滑的学习率变化, 更容易达到收敛到最优的目的。但对于 MNIST 分类任务来说, 由于任务简单, 即使采用固定适当的学习率也能很快达到收敛到最优的目的, 所以加入学习率调度的改善不明显。2. 加入 Dropout 正则化方法后能较好地降低模型过拟合的风险, 但对模型收敛速度和准确率提升方面影响不大

**展望:** 在更为复杂的分类任务中, 加入学习率调度后训练模型的收敛速度会更快并且在多轮 epoch 训练过后, 更细致的学习率调度会使模型更容易达到最优效果, 准确率能更高。在加入 Dropout 正则化方法后, 能更好的使模型在多轮次的训练过程中, 降低过拟合风险, 实现在训练集上表现好的同时, 在验证集和测试集上也能有很好的表现。

## 6 附录

**Github 代码仓库:** [https://github.com/FYK2004/Neural\\_Network\\_PJ1](https://github.com/FYK2004/Neural_Network_PJ1)

**模型权重下载地址:**

[https://drive.google.com/drive/folders/1F\\_fenFCldf5rCf7mcZrWqws8XxPugWL5?usp=drive\\_link](https://drive.google.com/drive/folders/1F_fenFCldf5rCf7mcZrWqws8XxPugWL5?usp=drive_link)